upwork™

BLOGER: Bi-Level Optimization for Generative Recommendation

Bridging Tokenization and Generation

Yimeng Bai, Chang Liu, Yang Zhang, Dingxian Wang, Frank Yang,
Andrew Rabinovich, Wenge Rong, Fuli Feng

Outline

- Background
- Methodology
- Experiment
- Conclusion

Background

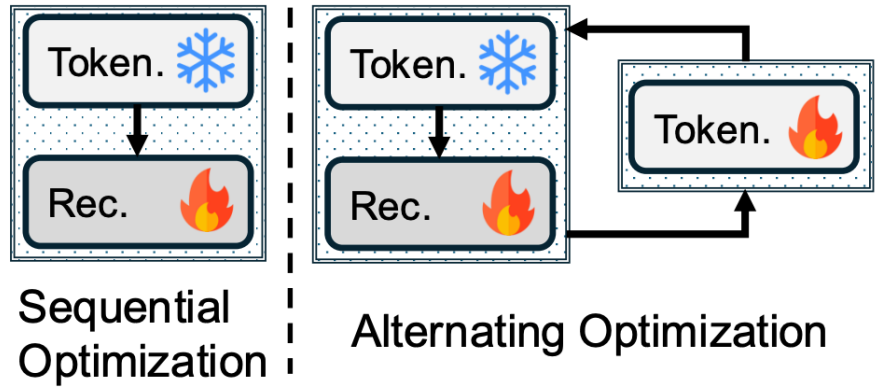
➢ Generative Recommendation (SID-based)

- Two coupled stages in GR
  - **Autoregressive generation**: instead of matching each candidate item, the model autoregressively generates the next item's identifier
  - **Item tokenization**: map items into **compact discrete representations**
    - Example: RQ-VAE / RQ-Kmeans for generative recommendation
- Critical role of item tokenization
  - Support the **direct generation** of items within the pre-defined token space
  - Improve **scalability** by replacing a large item vocabulary with small shared codebooks
  - Support generalization to **new / long-tail** items through content-derived identifiers

Background

➢ Separate Training Creates an Alignment Gap

- Existing works typically treat item tokenization and autoregressive generation as **independent** components
  - **Sequential optimization:** the tokenizer is first trained independently to produce static item IDs, which the recommender then uses for generation
  - **Alternating optimization:** the tokenizer and recommender are updated iteratively
  - **Decoupled** optimization trains the tokenizer **without recommendation guidance**, yielding misaligned identifiers that hurt recommendation performance.



Background

➢ Our Key Insight: Model the Mutual Dependence

- Root cause: separate training breaks the **interdependence**
  - A better tokenizer → better item representations → more accurate next-item prediction
  - The recommender needs recommendation-aligned identifiers to model user preference
- **Our key insight**
  - The tokenizer's efficacy is directly reflected in the recommender's performance
- Reformulate it as a **bi-level optimization** problem
  - Inner level trains the recommender on tokenized inputs
  - Outer level optimizes the tokenizer using both tokenization loss and recommendation loss

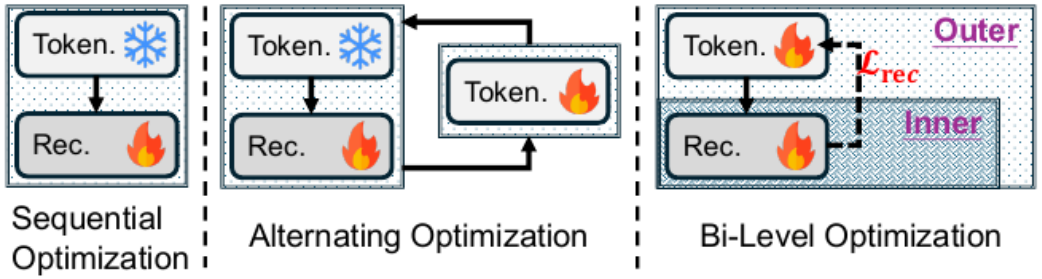
Methodology

➢ Bi-Level Optimization Formulation

- Prior strategies decouple tokenization and generation
  - **Sequential**: tokenizer frozen after pre-training, no recommendation guidance
  - **Alternating**: iterative updates via hand-crafted auxiliary alignment losses
- **BLOGER — unified bi-level objective**

$$\min_{\phi} \quad \mathcal{L}_{\text{rec}}(\mathcal{T}_\phi, \mathcal{R}_{\theta^*}) + \lambda \mathcal{L}_{\text{token}}(\mathcal{T}_\phi) \qquad \text{(outer)}$$

$$\text{s.t.} \quad \theta^* = \arg\min_{\theta} \mathcal{L}_{\text{rec}}(\mathcal{T}_\phi, \mathcal{R}_\theta). \qquad \text{(inner)}$$

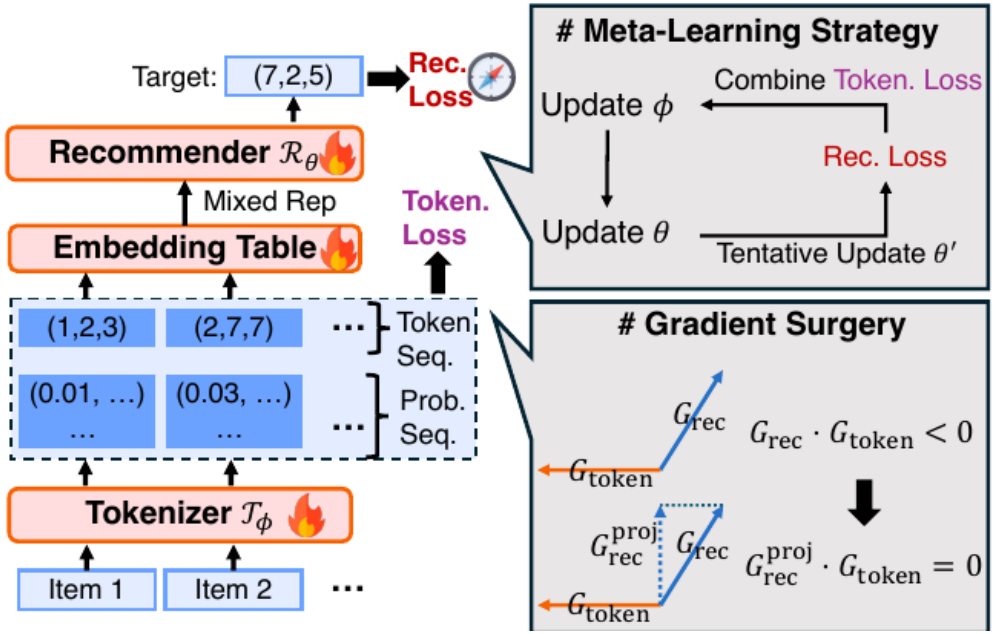


- Recommendation loss directly supervises the tokenizer — no auxiliary objective needed

Methodology

➢ BLOGER Framework Overview

- Encoder-decoder recommender (T5) + RQ-VAE tokenizer, trained jointly
- **Mixed representation** over codebook probabilities makes the recommendation loss differentiable w.r.t. the tokenizer
- **Meta-learning strategy** enables efficient bi-level optimization
- **Gradient surgery** extracts the beneficial, conflict-free component of the recommendation gradient



Methodology

➢ Item Tokenizer: Residual Quantization

- Represent each item as **L discrete tokens** via RQ over L codebooks

  Soft assignment: $$P(k|\boldsymbol{v}_l) = \frac{\exp\left(-\|\boldsymbol{v}_l - \boldsymbol{e}_l^k\|^2\right)}{\sum_{j=1}^{K} \exp\left(-\|\boldsymbol{v}_l - \boldsymbol{e}_l^j\|^2\right)},$$

  Hard token: $c_l = \arg\max_k P(k|\boldsymbol{v}_l)$, ; residual: $\boldsymbol{v}_l = \boldsymbol{v}_{l-1} - \boldsymbol{e}_{l-1}^{c_{l-1}}$. $\tilde{\boldsymbol{r}} = \sum_{l=1}^{L} \boldsymbol{e}_l^{c_l}$,

  Reconstruct semantic embedding from quantized code: $\tilde{z} = \text{Decoder}_{\mathcal{T}}(\tilde{\boldsymbol{r}})$

- **Tokenization loss** = reconstruction + commitment with stop-gradient

$$\mathcal{L}_{\text{token}}(\mathcal{T}_\phi) = \sum_i (\|\tilde{\boldsymbol{z}} - \boldsymbol{z}\|^2 + \sum_{l=1}^{L} \|\text{sg}[\boldsymbol{v}_l] - \boldsymbol{e}_l^{c_l}\|^2 + \beta \|\boldsymbol{v}_l - \text{sg}[\boldsymbol{e}_l^{c_l}]\|^2),$$

Methodology

➢ Mixed Representation for Differentiability

- Discrete tokenization is **non-differentiable** — blocks gradient flow to the tokenizer
  - Soft embedding: s_l^t = weighted average of vocabulary embeddings E_V over padded probabilities
  - **Mixed representation:** $\boldsymbol{o}_l^t = \boldsymbol{s}_l^t + \mathrm{sg}[\boldsymbol{h}_l^t - \boldsymbol{s}_l^t],$
  - Keeps the hard embedding's value, but lets gradients flow only through the soft part
- **Seq2Seq training on the T5 recommender:**

$$\boldsymbol{H}^E = \mathrm{Encoder}_{\mathcal{R}}(\boldsymbol{E}^X). \quad \boldsymbol{H}^D = \mathrm{Decoder}_{\mathcal{R}}(\boldsymbol{H}^E, \boldsymbol{E}^Y).$$

$$\mathcal{L}_{\mathrm{rec}}(\mathcal{T}_\phi, \mathcal{R}_\theta) = -\sum_{l=1}^{L} \log P(Y_l | X, Y_{<l}), \quad \text{(autoregressive next-item generation)}$$

Methodology

➢ Meta-Learning Strategy

- Nested optimization is hard to solve directly → adopt a **MAML-style meta-learning** strategy
- **Inner update (recommender):**

$$\theta \leftarrow \theta - \eta_{\mathcal{R}} \nabla_{\theta} \mathcal{L}_{\text{rec}}(\mathcal{T}_{\phi}, \mathcal{R}_{\theta}),$$

- **Meta-train — tentative recommender update (tokenizer fixed):**

$$\theta' = \theta - \eta_{\mathcal{R}} \nabla_{\theta} \mathcal{L}_{\text{rec}}(\mathcal{T}_{\phi}, \mathcal{R}_{\theta}).$$

- **Meta-test — update tokenizer with the tentative recommender:**

$$\phi \leftarrow \phi - \eta_{\mathcal{T}} (\nabla_{\phi} \mathcal{L}_{\text{rec}}(\mathcal{T}_{\phi}, \mathcal{R}_{\theta'}) + \lambda \nabla_{\phi} \mathcal{L}_{\text{token}}(\mathcal{T}_{\phi})),$$

Back-propagation: $\nabla_{\phi} \mathcal{L}_{\text{rec}}(\mathcal{T}_{\phi}, \mathcal{R}_{\theta'}) \rightarrow \theta' \rightarrow \nabla_{\theta} \mathcal{L}_{\text{rec}}(\mathcal{T}_{\phi}, \mathcal{R}_{\theta}) \rightarrow \phi.$

Methodology

➢ Gradient Surgery

- Outer update mixes two gradients: $G_{rec} = \nabla_\phi \mathcal{L}_{rec}$ and $G_{token} = \nabla_\phi \mathcal{L}_{token}$
- **Observation: gradient conflict** (negative cosine) affects up to **90%** of parameter groups
- **Gradient surgery** — when $G_{rec} \cdot G_{token} < 0$, project out the conflicting component:

$$G_{\text{rec}}^{\text{proj}} = G_{\text{rec}} - \frac{G_{\text{rec}} \cdot G_{\text{token}}}{\|G_{\text{token}}\|^2} G_{\text{token}},$$

  - After projection $G_{rec} \cdot G_{token} = 0$, — conflict removed, benefit retained
- Applied **per named parameter group** for finer-grained conflict mitigation

Methodology

➢ Training Algorithm & Complexity

- **Training loop (Algorithm 1):**
  - Each step: update recommender θ via Eq. (14)
  - Every M steps: tentative update θ', compute G_{rec} / G_token, apply surgery, update tokenizer ϕ
  - Delaying ϕ updates lets the recommender give stable gradients to the tokenizer
- **Complexity:** $O(TLKd + T^2d + Td^2)$ — same order as TIGER
  - **Inference complexity is identical to TIGER** — no significant additional overhead

Experiment

➢ Setup: Datasets & Metrics

- Three Amazon Review subsets, 5-core filtering, leave-one-out split, max length 20
- Metrics: Recall@K and NDCG@K (K = 5, 10), full ranking, constrained Trie decoding, beam search with 20
- Recommender: T5, 4+4 layers, dim 128; RQ-VAE: 4 codebooks × 256 codes, dim 32

Dataset	#Users	#Items	#Interaction	Sparsity	AvgLen
Beauty	22,363	12,101	198,502	99.93%	8.87
Instruments	24,772	9,922	206,153	99.92%	8.32
Arts	45,141	20,956	390,832	99.96%	8.66

Experiment

➢ Overall Performance (RQ1)

- Compared against 11 baselines: MF, LightGCN, Caser, GRU4Rec, HGN, SASRec, P5, TIGER, OneRec, LETTER, ETEGRec

Dataset	Metric	MF	LightGCN	Caser	GRU4Rec	HGN	SASRec	P5	TIGER	OneRec	LETTER	ETEGRec	BLOGER
Beauty	R@5	0.0226	0.0208	0.0110	0.0229	0.0381	0.0372	0.0400	0.0384	0.0408	0.0424	0.0413	**0.0444**
	N@5	0.0142	0.0127	0.0072	0.0155	0.0241	0.0237	0.0274	0.0256	0.0258	0.0264	0.0271	**0.0293**
	R@10	0.0389	0.0351	0.0186	0.0327	0.0558	0.0602	0.0590	0.0609	0.0621	0.0612	0.0632	**0.0654**
	N@10	0.0195	0.0174	0.0096	0.0186	0.0297	0.0311	0.0335	0.0329	0.0332	0.0334	0.0342	**0.0361**
Instruments	R@5	0.0456	0.0722	0.0425	0.0559	0.0797	0.0709	0.0809	0.0865	0.0879	0.0872	0.0878	**0.0882**
	N@5	0.0376	0.0608	0.0348	0.0459	0.0676	0.0565	0.0695	0.0736	0.0743	0.0747	0.0745	**0.0753**
	R@10	0.0511	0.0887	0.0528	0.0702	0.0967	0.0922	0.0987	0.1062	0.1073	0.1082	0.1079	**0.1100**
	N@10	0.0394	0.0661	0.0381	0.0505	0.0731	0.0633	0.0751	0.0799	0.0801	0.0814	0.0810	**0.0822**
Arts	R@5	0.0473	0.0464	0.0571	0.0669	0.0667	0.0758	0.0724	0.0807	0.0828	0.0841	0.0860	**0.0879**
	N@5	0.0271	0.0244	0.0407	0.0518	0.0516	0.0632	0.0607	0.0640	0.0653	0.0675	0.0687	**0.0703**
	R@10	0.0753	0.0755	0.0781	0.0834	0.0910	0.0945	0.0902	0.1017	0.1067	0.1081	0.1084	**0.1108**
	N@10	0.0361	0.0338	0.0474	0.0523	0.0595	0.0693	0.0664	0.0707	0.0730	0.0752	0.0759	**0.0777**

Experiment

➢ Ablation Study (RQ2)

- Systematically disable each design element on Beauty

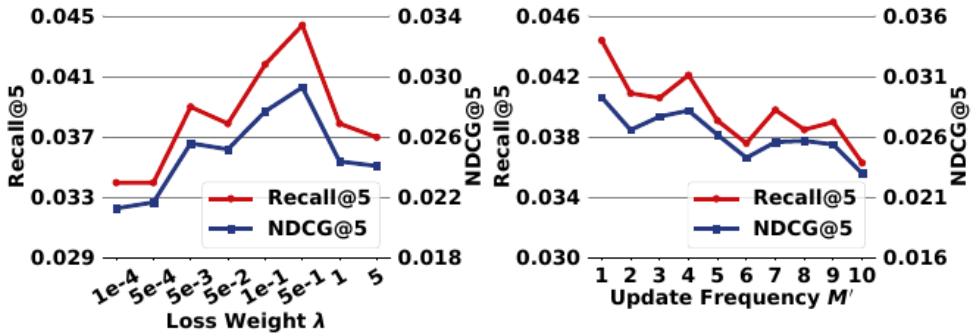
Method	R@5	N@5	R@10	N@10
BLOGER	**0.0444**	**0.0293**	**0.0654**	**0.0361**
w/ LETTER	0.0489	0.0322	0.0732	0.0400
w/o GS	0.0329	0.0209	0.0530	0.0274
Joint	0.0418	0.0272	0.0637	0.0342
Joint w/ GS	0.0412	0.0273	0.0631	0.0343
Fixed	0.0384	0.0256	0.0609	0.0329

- **w/o GS**: removing gradient surgery causes the sharpest drop — conflict-free gradients are essential
- **Bi-level** beats both Joint and Fixed; pairing with LETTER further improves — backbone-agnostic

Experiment

➢ Hyper-Parameter Analysis (RQ3)

- **Loss weight λ** balances recommendation vs. tokenization supervision at the outer level
  - Too small → weak constraint disrupts identifier structure; too large → recommendation loss cannot supervise
  - $\lambda = 0.5$ strikes the best balance
- **Update frequency M'** of the tokenizer is stable for $M' \in [1, 4]$
  - Too frequent → recommender under-optimized → unreliable meta-gradients



Experiment

➢ Time Efficiency (RQ4)

- Average per-epoch training / testing time vs. TIGER on identical NVIDIA RTX 3090

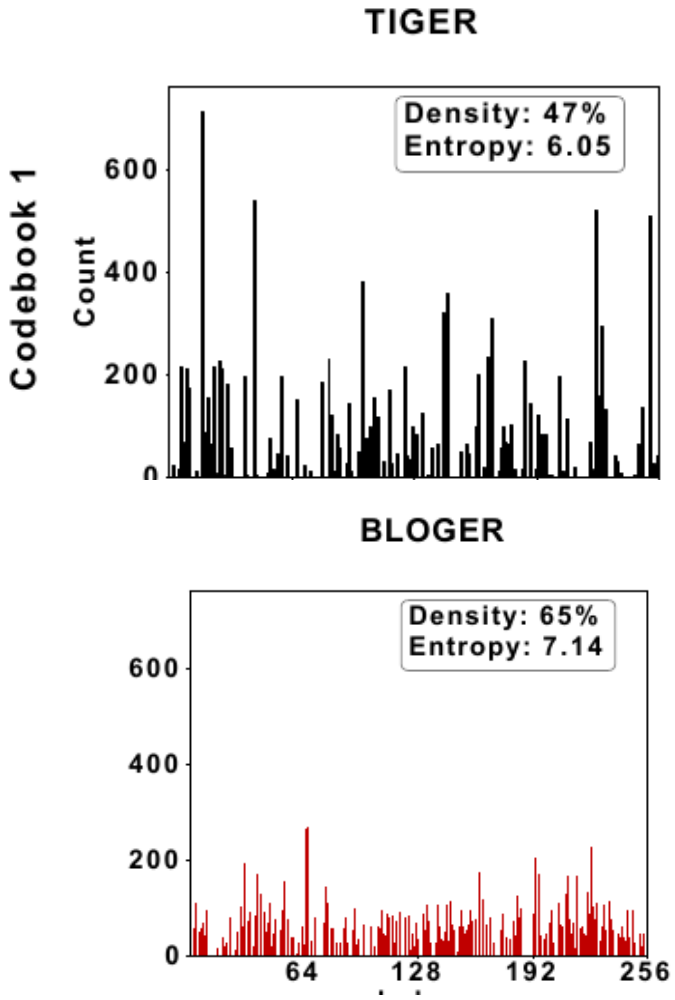
Method	Train (s/epoch)	Test (s/epoch)
TIGER	23.7	66.7
BLOGER	**26.6**	**66.9**

- Slight training increase, **near-identical inference cost**
- Bi-level optimization adds minimal overhead — confirms the complexity analysis

Experiment

➢ Codebook Utilization Analysis (RQ5)

- Analyze first-level codebook utilization on Beauty (TIGER vs. BLOGER)
  - **Density 47% → 65%**, Entropy 6.05 → 7.14
  - First level carries most information (residual op); higher levels differ little
- **Recommendation loss acts like a compass**
  - It reveals hidden collaborative signals, correcting text-only bias



Conclusion

➢ Summary

- Reformulated generative recommendation as a **bi-level optimization** problem
  - Explicitly captures the interdependence between tokenizer and recommender
- BLOGER solves it efficiently via **meta-learning** + **gradient surgery**
  - Recommendation loss directly aligns tokenization with the downstream objective
  - **No significant additional computational overhead**
- **Gradient surgery is critical** — up to 90% of parameter groups exhibit gradient conflict
- Gains primarily stem from **improved codebook utilization**

Thanks